



Agentic AI Foundation

# AGNTCon + MCPCon China

Where the agentic stack is being built.

## 从单兵到军团： 基于 A2A 协议构建高可扩展的 多智能体分布式系统

AI Evangelist, Google Cloud / Sylph Lin



# Agenda

WHY A2A?



HOW A2A?



# Agenda

---

WHY A2A?



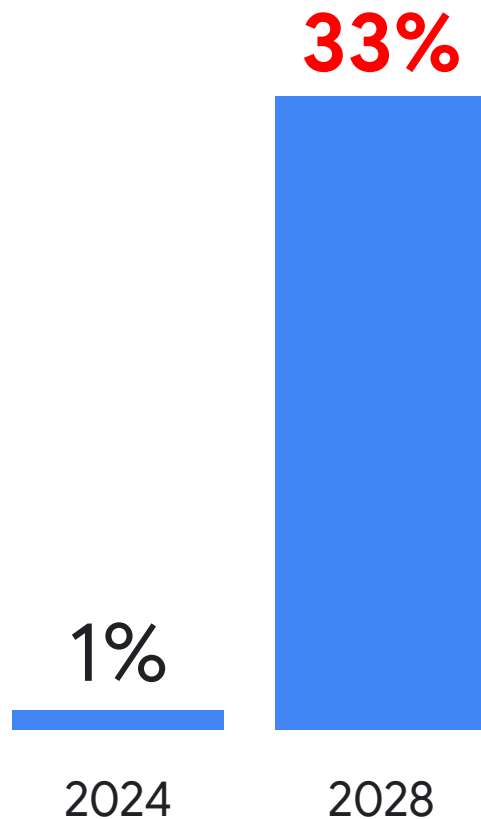
HOW A2A?



具备智能体功能的  
企业软体比例

到2028年，具备智能体功能的企业软体比例将从2024年的不足1%提升至**33%**，从而将**15%**的日常工作决策转变为自主执行

*Gartner: Intelligent Agents in AI Really Can Work Alone. Here's How.*



已使用生成式 AI 的企业中，**52%** 的高阶主管表示，他们已将智能体实际导入生产环境中

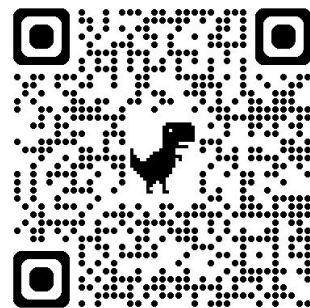
用在哪里？

49% 客户服务

46% 行销或安全运营

45% 技术支持

43% 的产品创新或生产力提升与研发



Google Cloud, [The ROI of AI](#), 2025

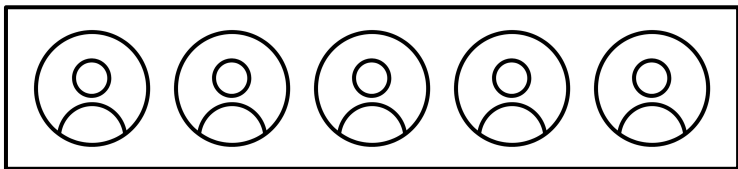
在智能体的时代

个人能力的上限  
被大幅度的提高了

# 团队的变化

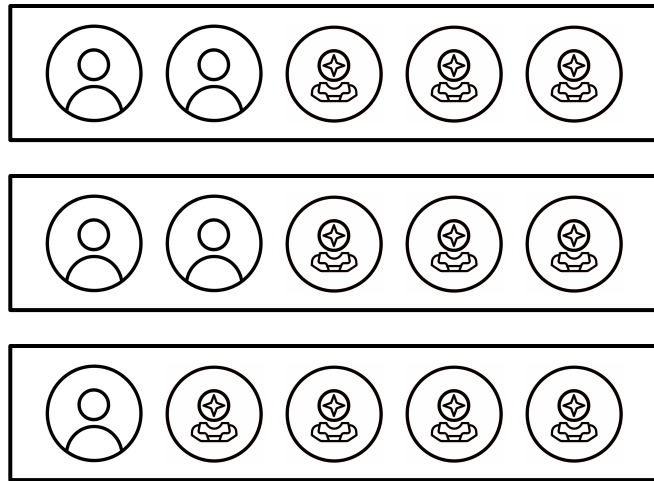
## 现在的团队

以人类成员为主的传统分工模式



## 未来的团队

人类与多智能体协同的全新形态



# 为什么需要A2A多智能体系统



**单一智能体系统**  
(单兵作战)



**协同子智能体系统**  
(中央集权)



**A2A 多智能体系统**  
(分散自治)

## 迁移要件

- 工具选择过载
- 对话语境污染
- 工作流程失控

## 迁移要件

- 信任边界失效
- 共享状态越权
- 自主博弈缺失



# 单一智能体系统 to 协同子智能体系统



## 工具選擇過載

当单一智能体承担全能职责而挂载过多工具时, 工具描述过多会稀释注意力, 引发工具误判与参数幻觉

解决「职责分工」



## 对话语境污染

长程对话中原始资料、中间工具输出与错误推导全部混入同一 Context, 导致遗忘核心规范并放大推理偏误

解决「记忆隔离」



## 工作流程失控

純 LLM 自主 ReAct 循環缺乏硬性狀態約束, 容易在非核心步驟無限發散或陷入無窮探索死迴圈

解決「確定性控制」

# 协同子智能体系统 to A2A 多智能体系统



## 信任边界失效

当协同对象延伸至外部厂商、不同部门或独立App时, 无法依赖统一的内部调用或预设的完全信任环境

解决「跨域通讯」



## 共享状态越权

依赖全知全能的中央共享状态与传递资料, 会迫使敏感机密全域曝光, 违反资料主权与最小权限原则

解决「资料隐私」



## 自主博弈缺失

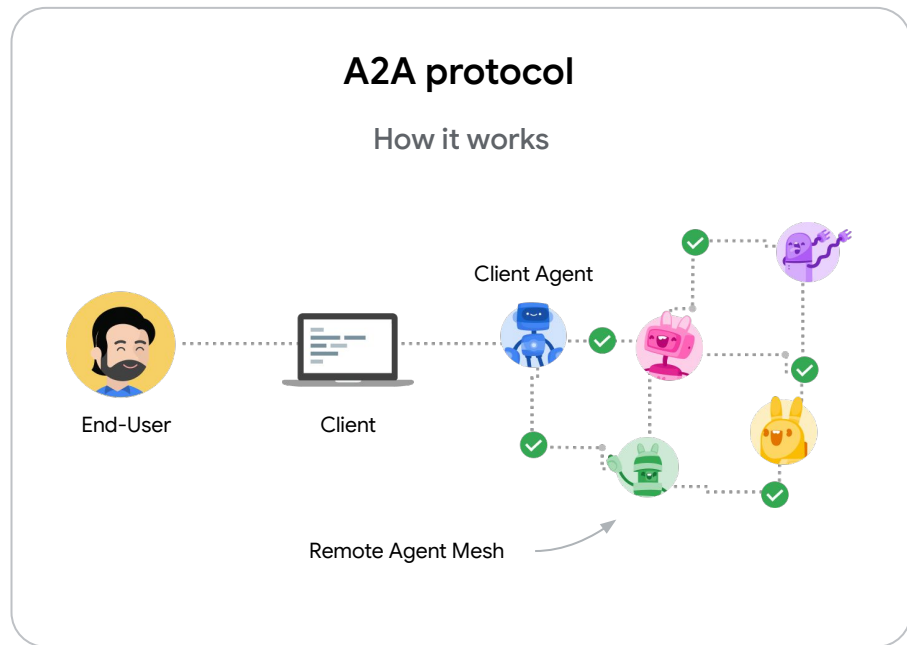
中央编排预设子节点无条件服从派工, 面对买卖议价、资源抢占或合规审查等利益冲突场景时, 无法实现对等谈判与自主拒绝

解决「利益冲突」

# Agent to Agent (A2A) Protocol

## A new standard for Agent-to-Agent collaboration is needed

- 专为处理代理协作的动态和长期运作特性而设计的开放协议
- 使用Agent Card来实现智能代理发现
- 设计时充分考虑了企业用途, 提供安全且可控的体验
- 基于现有的基础标准包含: HTTP, SSE, JSON-RPC



# A2A and MCP - 互补而非竞争

## MCP (Model Context Protocol)

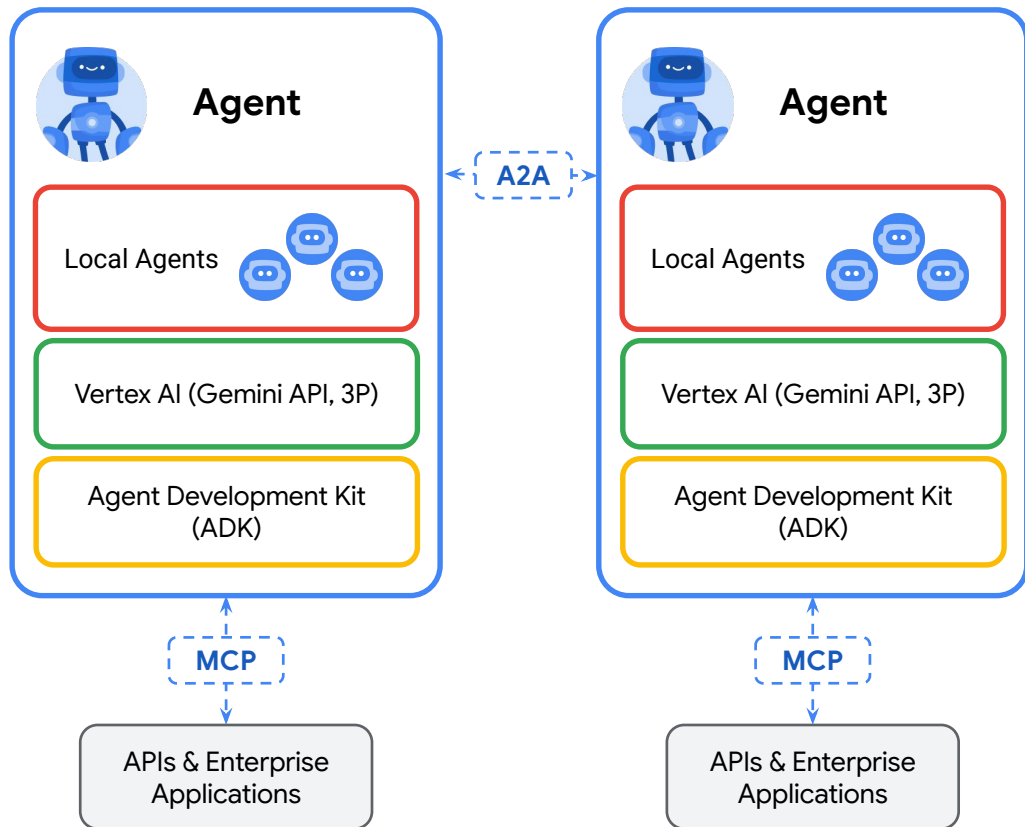
*for tools and resources*

- 透过结构化的输入/输出将代理连接到工具、API 和资源
- ADK 支援 MCP 工具, 支援多种 MCP 伺服器与代理程式配合使用

## A2A (Agent2Agent Protocol)

*for agent-agent collaboration*

- 不同代理之间无需共享记忆、资源和工具即可进行动态、多模式通信
- 由社群推动的开放标准
- 可使用 ADK、LangGraph、Crew.AI 的范例



# Agenda

WHY A2A?



---

HOW A2A?



# A2A的基础沟通元件

## 输入

### Atent Card

每个 Agent 的公开规格书, 宣告能力、技能清单、授权认证与端点

/.well-known/agent.json

## 运行

### Task

管理状态机与非同步生命周期

### Message

协同过程中的「语义交流」

Ex: Agent 间的提问、确认参数、回报目前进度

### Part

承载纯文字、JSON 数据或档案切片

## 输出

### Artifacts

协同完成后的「实体成果」

Ex: PDF 报表、音讯档案、结构化 JSON 资料、程式码

### Part

承载纯文字、JSON 数据或档案切片

# A2A 的运作流程

01

**Discovery &  
Negotiation  
发现与握手**

Client 透过 Agent Card 获取 Server 的能力定义与鉴权要求，组装合规的请求

02

**Task  
Instantiation  
任务实例化**

Client 发起 Task 请求，Server 验证权限与参数后返回 Task ID，标记为执行中

03

**Execution &  
Event Streaming  
非同步执行与事件推播**

Server 透过 SSE (Server-Sent Events) 或 Webhook 向 Client 回传状态转移、即时进度更新或提问请求

04

**Resolution &  
Delivery  
任务结案与交付**

Server 将任务标记为 completed，将最终 Artifact 交付给 Client，关闭状态机

# A2A 如何解决我们的问题？



## 信任边界失效

解决「跨域通讯」

Agent Card + 标准  
HTTP/REST/SSE 协议, 搭  
配 OAuth 身分鉴别, 原  
生跨域与跨实体运作



## 共享状态越权

解决「资料隐私」

Task 封装 + Artifacts 成  
果交付, 各节点内部资料  
库与推导过程彻底黑盒  
化



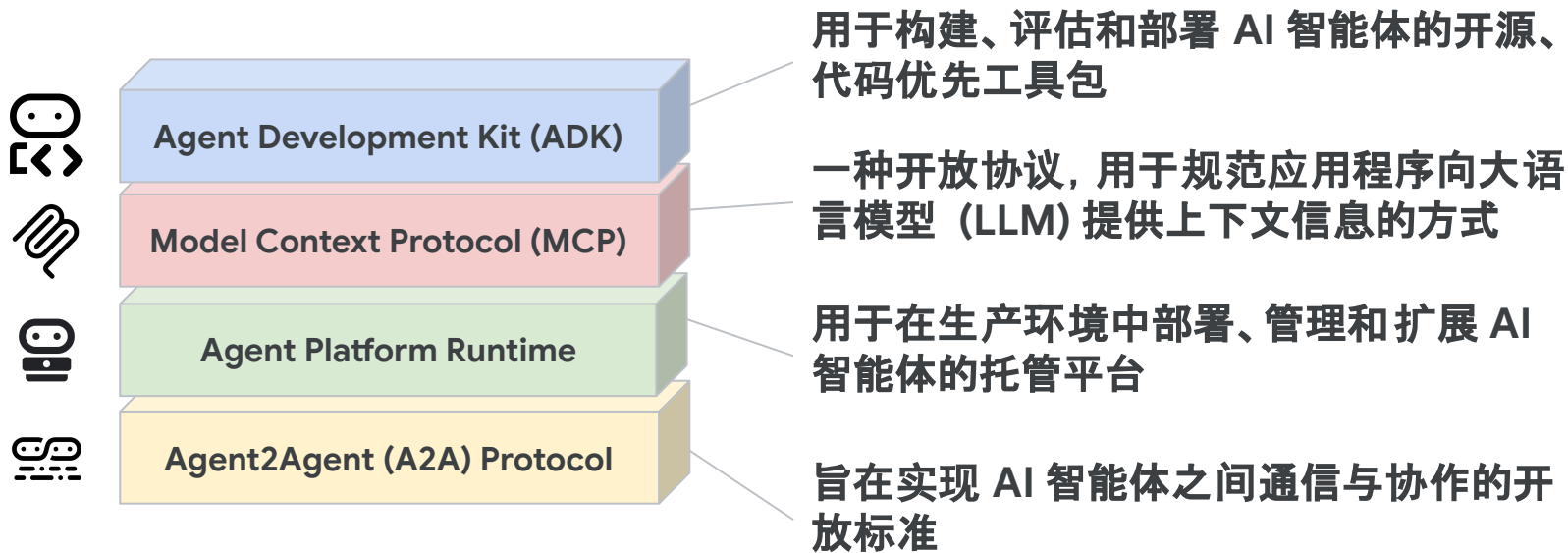
## 自主博弈缺失

解决「利益冲突」

非同步状态机(支援拒  
绝、提议、等待外部批  
准), 节点可依自身策略  
接受或退回 Task



# 可能的智能体技术栈



# 利用ADK协助开发A2A Agent (ADK 2.0 Update)



## Graph-based workflows (Python)

建构确定性的工作流程，对任务的路由与执行方式有更强的掌控力



## Dynamic workflows (Python)

在执行过程中，根据即时上下文、用户反馈及工具库，自主决定下一步动作



## Collaborative agents (Python)

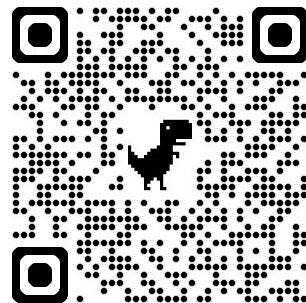
利用协调者与多个子代理的协同工作，建构复杂的智能体架构



## Agent Skills support in ADK

现已支援Agent Skills(Python、Go、TS)，使AI Agent能够动态执行特定领域技能

体验 ADK python 开发



<https://github.com/google/adk-python>

# 利用 agents-cli 帮你快速搭建並部署A2A Agents



## 针对 ADK 封装 Skills

封装了开发、评估、部署发布能力，大幅度缩短从开发到生产上线的时间



## 完整的 CLI 命令行工具

方便开发者或 Coding Agent 使用，一键实现高效的开发与管理流程



## 支援主流 Coding Agents

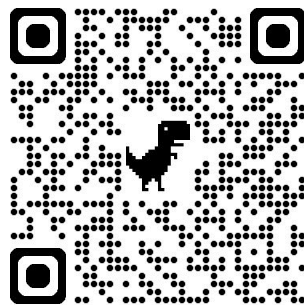
引导 Coding Agent 了解开发流程，原生支持 Antigravity, Gemini CLI, Claude Code 等



## 自动化 Eval 评估框架

提供离线与线上评估，支持自动编写并运行评估任务，持续迭代直至满足标准

掃描體驗 agents-cli



<https://github.com/google/agents-cli>